

# Smart Lockbox using Firebase and ESP32 for Secure Physical Key Storage with Real-Time Return Monitoring

Venkata Sriram Topalli  
Vellore Institute of Technology  
Vellore, India  
23BCE0441

**Abstract**—Old ways of handling keys in real estate cause problems because they do not track who opens what or when. Instead of using basic metal boxes, this project introduces a digital alternative built with connected tech. At its heart sits an ESP32 chip, running the whole setup like a small brain. To know if keys are inside, it uses sound waves from an HC-SR04 sensor to sense their presence. Opening happens through movement - a tiny motor turns parts to release locks only when needed. When something changes, a loud beep sounds right at the box to signal activity. Information flows instantly into a cloud database powered by Firebase, linking actions across devices. A personalized website updates live, showing every event tied to each container. So everything stays linked - hardware, data, and user view - all working together without delay. One thing stands out here: instead of leaning on Bluetooth or generating PINs without connection, this early version tracks whether keys get brought back. Not just grabbed safely - put away on schedule too. What makes it tick gets tested closely - the design backbone, code that reacts to triggers, two-brain processing style, how adjustments play out in real conditions. Results show something affordable, built different when needed, handling entry control from afar.

**Index Terms**—ESP32, Firebase, Access Control, Ultrasonic Sensor, Cyber-Physical Systems

## I. INTRODUCTION

Back in the day, real estate relied on slow, hands-on tasks. Now, tech like PropTech and IoT is shifting how things work. At the heart of it? Controlling who gets into properties - especially empty ones spread out across locations. Getting keys to the right people securely, quickly, and traceably has always been tricky. Old-school metal lockboxes offer zero record of who came or when they showed up. Without logs, owners are left guessing about visits and time spent inside. That gap causes headaches and opens doors to risk.

A locked box with brains finds its place where old systems won't budge. Not tied to any one setup, it moves easily, holding real keys when changing doors isn't allowed or runs too deep into pockets. Some brands already out there - solid names, sure - but their price tags loom large, and their tech plays only with itself, shutting out cheaper ways forward. When budgets tighten, those perks fade fast.

Out here begins a homemade Smart Lockbox built around an ESP32 chip tied to Firebase's live database. Not like old-school padlocks, yet cheaper than high-end smart models

found online. Digital approvals trigger actual lock movements through tight feedback loops linking code and mechanics. A twist comes when sensors check if keys are back inside - only then does the electronic handshake finish, easing worries about lost access tools. Inside this report: how circuits were wired, how programs run, logic patterns used, plus speed tests on the working model.

## II. LITERATURE REVIEW

Getting into access control lately means plenty of patents, lots of business moves, shifting slowly from old-school locks to digital checks. Take US Patent 20170178434A1 [1] - it shows lockboxes running on Raspberry Pi, using cell signals to scan IDs. Though tight on security, depending on private telecom setups and heavy computing makes each unit expensive and thirsty for power. Another example, CA2565505C [2], spots keys through tamper-detecting circuits. Yet it insists on locked-down parts, blocking outside tools from working with it.

Out there, gadgets such as the Master Lock 5441EC [5] and the Igloohome Smart Keybox 3 [6] lead the pack. Instead of needing constant web links, they mostly use Bluetooth Low Energy or algoPIN methods to let people in. Yet here's the catch - none of these widely sold units watch what happens inside once closed. Opening gets checked, sure, though whether the actual key goes back in? That part slips through the cracks every time. Now picture this: an early version built around the ESP32 chip steps in, pairing ultrasound detection with a transparent, low-cost system living online.

## III. SYSTEM ARCHITECTURE

When something happens, the Smart Lockbox reacts through three parts working together - hardware at the front, cloud storage nearby, then a web interface people see. Sitting in the physical unit, the ESP32 chip connects straight to an ultrasonic sensor that checks distance, alongside a small motor handling movement, also tied to a buzzer that sounds when triggered. Each piece listens for signals instead of running constantly, staying quiet until needed.

Inside the setup, Firebase Realtime Database links the ESP32 with a tailored web app. This structure runs like a loop where physical actions tie into digital controls, a core

TABLE I  
COMPARATIVE ANALYSIS OF PRIOR ART AND COMMERCIAL ACCESS SOLUTIONS

| Ref.        | Patent / Product             | Core Technology & Features                                         | Key Limitations                                                                                       |
|-------------|------------------------------|--------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------|
| [1]         | US Patent 20170178434A1      | Raspberry Pi MCU, 3G/4G cellular, ID card scanning.                | High unit cost; relies on proprietary cellular networks; no key presence detection.                   |
| [2]         | CA Patent 2565505C           | RFID/barcode with tamper-sense loops.                              | Closed ecosystem preventing third-party API integration.                                              |
| [3]         | EP Patent 2071522A1          | Lockbox key with callback feature.                                 | Complex mechanical integration; lacks cloud connectivity.                                             |
| [4]         | WO Patent 2007057575A1       | Electromechanical safety lock.                                     | No remote provisioning or dynamic access codes.                                                       |
| [5]         | Master Lock 5441EC           | Bluetooth Low Energy (BLE) authentication.                         | No real-time interior monitoring; no key-return verification.                                         |
| [6]         | Igloohome Keybox 3           | Algorithmic PIN generation (algoPIN) for offline access.           | Lacks real-time cloud synchronization; cannot verify physical key return.                             |
| <b>Ours</b> | <b>Smart Lockbox (ESP32)</b> | <b>Firestore Realtime DB, Ultrasonic sensing, Wi-Fi telemetry.</b> | <b>High power consumption due to continuous Wi-Fi synchronization; lacks offline access fallback.</b> |

principle of modern IoT cyber-physical systems [10]. Instead of standalone operations, decisions made through the website set lock conditions - like choosing "UNLOCK." Those choices land straight into Firebase. Over Wi-Fi, the ESP32 keeps checking for changes in that data. Each update pulls it one step closer to responding in real time. Right after spotting a shift, the tiny computer triggers movement through the small motor while at the same time checking how far the key sits using sound waves. What it finds from that check gets sent straight into the online database so the website shows what's actually happening - whether the box is open or shut, whether the key is really there - not just what was asked earlier.

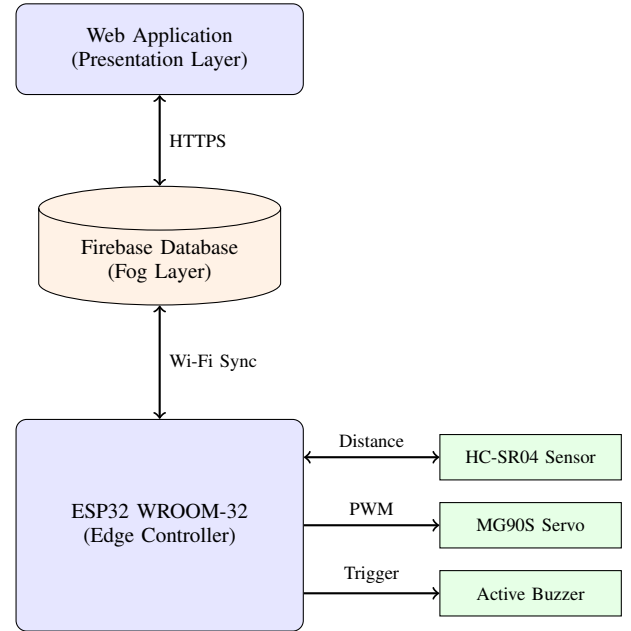


Fig. 1. Three-tier cyber-physical architecture demonstrating secure data flow from the web interface down to edge hardware actuation.

Old-school relational databases usually can't keep up with fast IoT systems - they're stiff and sluggish. Instead, this setup uses Firebase's NoSQL cloud platform [8], where info lives in a single sprawling JSON structure. Because of that layout, the ESP32 grabs updates without delay. Inside, the data splits across three main working sections. Right there, the Lockbox holds real-time status - servo switches as true or false, while numbers record how far objects are via ultrasound. Over in another corner, Buzzer keeps its own count until warnings fire, keeping alarms off the main path of entry checks. Happening at once, Requests plus Preapproval act like guarded notebooks. A keyholder makes a pass, drops it into Preapproval first. Only after someone proves who they are does that detail move - sliding into Requests to stay forever, clear to review later. As depicted in the visual appendix (Fig. 3), this exact JSON data tree can be seen in real-time operation, with Fig. 3a illustrating the active Lockbox and Buzzer nodes, Fig. 3b showing the permanent Requests history log, and Fig. 3c displaying the temporary Preapproval state.

#### IV. HARDWARE IMPLEMENTATION

Down inside sits an ESP32 WROOM-32 chip [7], picked because it handles wireless signals well and offers plenty of pins for connections. Running things: four rechargeable 1.2-volt cells wired up, giving a total of 4.8 volts at 2800 milliamp-hours. Voltage jumps get absorbed by a 1000-microfarad capacitor so the main processor does not restart unexpectedly when motors kick on. Instead of gears or bolts, a small TowerPro MG90S servo turns nearly half a circle to slide the lock into place - or pull it back. A small sound device made by ADIY connects through shared grounding and three 10kΩ resistors, sounding out nearby when something happens - say, a warning after waiting too long. The system checks if the key sits inside using an HC-SR04 sensor [9] that tracks how far it is from the bottom space.

Prior to building the actual system, the electrical properties were first tested with software-based circuit simulators. The initial testing process was vital for ensuring the proper power distribution. Through the simulation of the large current requirement of the servo motor, it was proven that the need for adding a 1000 microfarad capacitor between the power lines was essential. The use of the capacitor was meant to prevent a brownout from resetting the ESP32 controller chip.

Bouncing sound waves form the core of how it senses objects. From the HC-SR04, a high-pitched burst at 40 thousand hertz shoots out, then waits for what bounces back - either off the bottom or something inside like a key. Travel time of that returning ping gets turned into distance, thanks to how fast noise moves through air: about 343 meters every second.

$$Distance = \frac{Time_{travel} \times 343}{2} \quad (1)$$

When data arrives in microseconds, the tiny computer checks whether that span lines up with an open space - or something closer, meaning a key sits there.

## V. SOFTWARE AND ALGORITHM DESIGN

The embedded firmware controlling the edge controller was programmed using C++ and used the Arduino core specifically designed for the ESP32 chip. To cope with the complex network level functionality, this program uses special client libraries of Firebase that allow establishing secure and persistent SSL connections via Wi-Fi. In addition, the program does not employ manual bit-banging in order to achieve correct PWM frequency; rather, it uses special libraries for servos in order to ensure correct generation of PWM frequencies.

Most of the time, it just waits - until something changes. That change kicks off an update through Firebase, which acts like a live scoreboard everyone watches. Depending on who logs in - a Guest or an Owner - the screen shows different options. Rules built into the database decide what each person can see. Once someone real tries to get in, the website flips a switch inside the data space. Over on the hardware side, the ESP32 checks that spot again and again, never stopping. When the system catches an UNLOCK signal, it shifts gears fast: the servo releases the latch while kicking off a background check routine. Right then, the ultrasonic reader starts pinging nonstop, watching space like a hawk. Only when it sees the key back at its home spot will the unit allow itself to snap shut again. Should the object stay missing past the timeout mark, a sharp beep fires nearby - all while a record of the event shoots up to the remote log server.

Wasting CPU time is skipped by steering clear of endless loop checks. The design wakes up only when needed thanks to signal triggers. Whenever the HC-SR04 spots a shift in space, it fires off a pulse through the IE setup. That way, the chip does not sit frozen waiting around. One core manages live updates over Wi-Fi with Firebase while the other snaps to attention the moment the key moves.

One way to keep things secure involves handling online risks along with actual hardware actions. Because the setup

leans on Firebase Realtime Database Rules, changing states without approval becomes difficult. Inside the database, guest areas stay separate from owner zones by design. Only verified guest sessions may check their own entry status, thanks to encryption limits blocking broader access. Writing straight into key lock functions? That option does not exist for guests at all. Skipping locally made PIN codes removes a major weak spot - attackers can't just guess passwords offline anymore. Instead, each time someone tries to open the lock, confirmation comes first through the active cloud server. Then, and only then, does the ESP32 send the needed PWM pulse to move the servo.

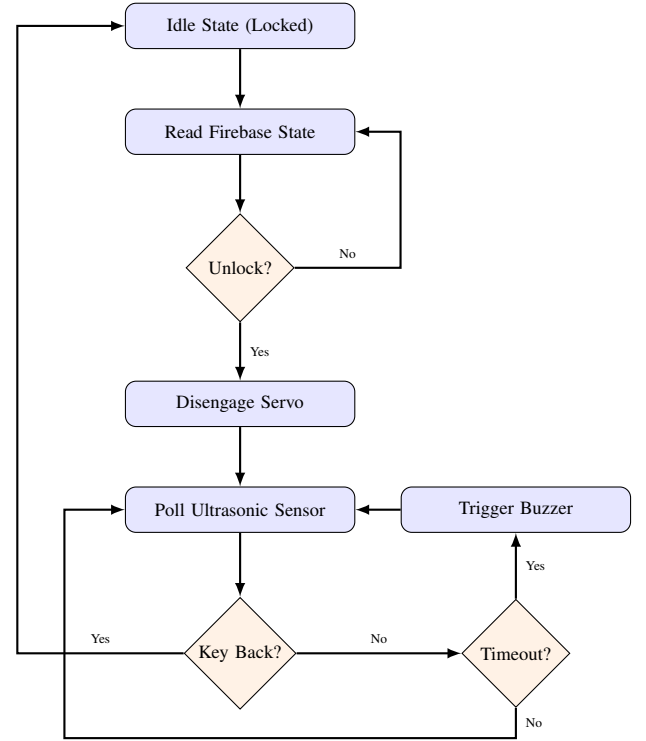


Fig. 2. Algorithmic state machine detailing the cyber-physical event loop and key-return verification process.

## VI. RESULTS AND PERFORMANCE METRICS

A working model showed how software and machinery connect smoothly, syncing live across online interface and actual parts. Not much lag happened when signals moved, proving the quick response method using wireless internet works well. The whole setup came in at around INR 11,480, shaped by smart budget choices. Basic gear stayed cheap - like the ESP32 chip costing INR 634, a reusable nickel-metal hydride battery pack priced INR 855, along with necessary detectors. Most money went into building strong digital infrastructure, taking up nearly INR 9,490 of the spending. A cheaper option than most ready-made versions sits here. Still, tests showed the constant check-ins with Firebase plus always-on Wi-Fi drained energy fast - pointing at better battery handling down the line.

Looking at how well the network runs mattered most when checking the system. To see how much bandwidth gets used, we studied the data sent through Firebase telemetry. Every

TABLE II  
SYSTEM BILL OF MATERIALS (BOM) AND PROTOTYPING COSTS

| Component                   | Quantity | Cost (INR)       |
|-----------------------------|----------|------------------|
| ESP32 WROOM-32 Module       | 1        | 634.00           |
| 1.2V 2800mAh Ni-MH Battery  | 4        | 855.00           |
| HC-SR04 Ultrasonic Sensor   | 1        | 75.00            |
| TowerPro MG90S Micro Servo  | 1        | 150.00           |
| ADIY Active Buzzer Module   | 1        | 20.00            |
| 1000 $\mu$ F Capacitor      | 1        | 20.00            |
| 10k $\Omega$ Resistors      | 3        | 10.00            |
| Breadboard & Jumpers        | 1        | 166.00           |
| Battery Pack Holder         | 1        | 40.00            |
| Secure Web App & Firebase   | 1        | 9,510.98         |
| <b>Total Prototype Cost</b> |          | <b>11,480.98</b> |

time the device shares its condition - like whether it is locked, what the sensor reads, and when it happened - it sends a small JSON message. Using 1KB as precisely 1000 bytes, just one update takes up that full kilobyte. So if someone opens or closes the box fifty times in a day, it uses no more than 50,000 bytes altogether. That amount shows it works fine even where internet access has tight limits or relies on mobile hotspots.

Figuring out how much power the earlier energy needs would take led to building a test model for the ESP32 edge device. A pack of Ni-MH batteries supplies 4.8 volts, holding 2800 milliamp-hours altogether. Since the chip keeps Wi-Fi live to sync with Firebase instantly, it pulls around 80 milliamperes on average while running. Battery runtime formulas help estimate when performance might drop off.

$$T_{hours} = \frac{C_{battery}}{I_{average}} = \frac{2800 \text{ mAh}}{80 \text{ mA}} = 35 \text{ hours} \quad (2)$$

With constant online activity, the setup lasts about 35 hours without shutting down - hinting that better idle modes should come next.

Additionally, system latency has been considered beyond bandwidth and power consumption as an important metric for achieving the desired cyber-physical experience for the user. RTT has been calculated from when the unlock button has been pressed on the web portal until the actual disengagement of the lock by the servo latch. Taking into consideration the latency due to cloud synchronization via Firebase, which typically varies between 300ms and 400ms using a regular Wi-Fi broadband connection, and the mechanical movement of the servo motor, the overall latency of the system never exceeded one second.

## VII. CONCLUSION

One big step forward happens when old lock ideas meet modern tech. Instead of costly custom systems, this device uses a small computer chip linked to online storage. What stands out most is how it watches keys come back using sensors - no trust needed, just proof. Only after that check does the system close the user's session. Next steps lean into battery life improvements. Slowing down processing when idle, reacting only when triggered, plus smarter timing for checking inputs - all these shape what comes next. Long runs

matter now, more than speed. Turning a working model into something people can rely on daily means making every joule count. Efficiency becomes the main goal, quietly. Not flash, just function lasting longer.

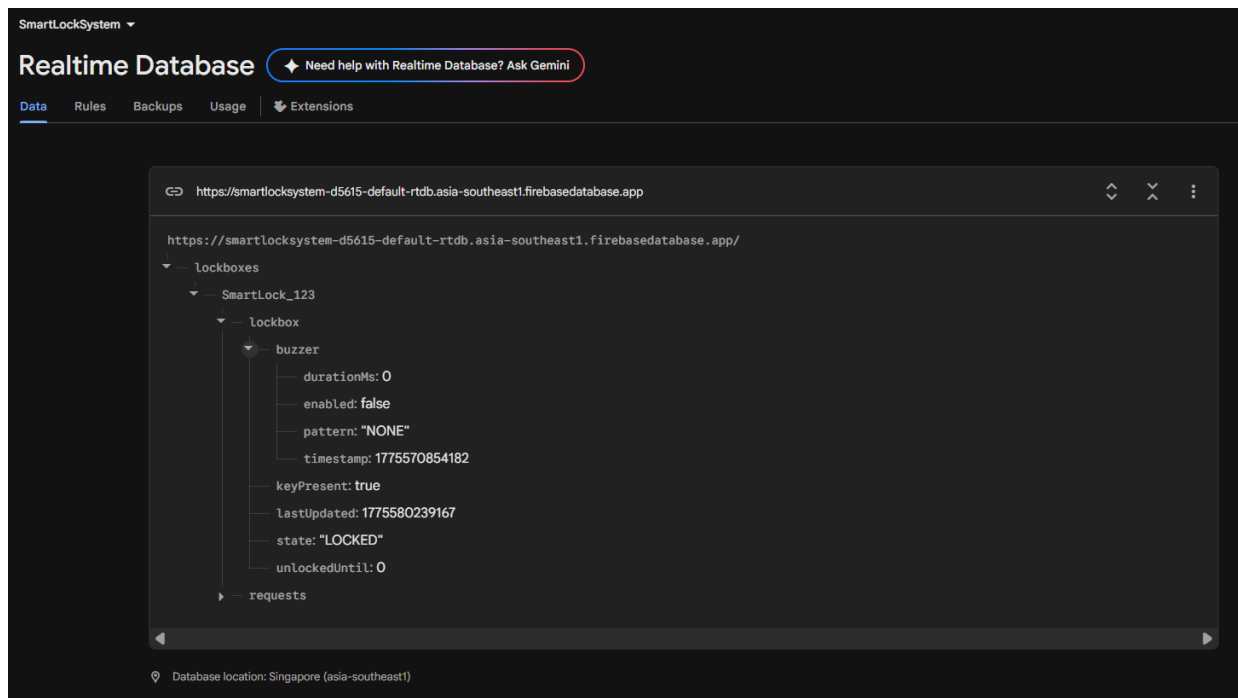
## REFERENCES

- [1] Firth et al., "Raspberry Pi MCU, 3G/4G cellular, solar power," U.S. Patent 20170178434A1, 2015.
- [2] Fisher (SentryLock LLC), "Key presence detection via tamper-sense loop," Canadian Patent 2565505C, 2005.
- [3] W. F. Larson, M. Harkins, C. Fale, T. L. Briskey, and J. Switzer, "Lockbox key with callback feature," European Patent EP2071522A1, 2009.
- [4] F. Giacomini, "Safety lock," WIPO Patent WO2007057575A1, 2007.
- [5] GoKeyless, "Master Lock 5441EC Bluetooth Wall-Mount Lock Box," 2026. [Online]. Available: <https://www.gokeyless.com/products/master-lock-5441ec>
- [6] Walmart, "Igloohome Smart Keybox 3," 2026. [Online]. Available: <https://www.walmart.com/ip/igloohome-Smart-Keybox-3/150494583>
- [7] Espressif Systems, "ESP32 Series Datasheet," v4.1, 2023. [Online]. Available: [https://www.espressif.com/sites/default/files/documentation/esp32\\_datasheet\\_en.pdf](https://www.espressif.com/sites/default/files/documentation/esp32_datasheet_en.pdf)
- [8] Google, "Firebase Realtime Database Documentation," Google Developers, 2024. [Online]. Available: <https://firebase.google.com/docs/database>
- [9] O. Science, "HC-SR04 Ultrasonic Sensor User Manual," ElecFreaks, 2011. [Online]. Available: <https://cdn.sparkfun.com/datasheets/Sensors/Proximity/HCSR04.pdf>
- [10] A. Al-Fuqaha, M. Guizani, M. Mohammadi, M. Aledhari, and M. Ayyash, "Internet of Things: A Survey on Enabling Technologies, Protocols, and Applications," *IEEE Communications Surveys & Tutorials*, vol. 17, no. 4, pp. 2347-2376, Fourthquarter 2015.

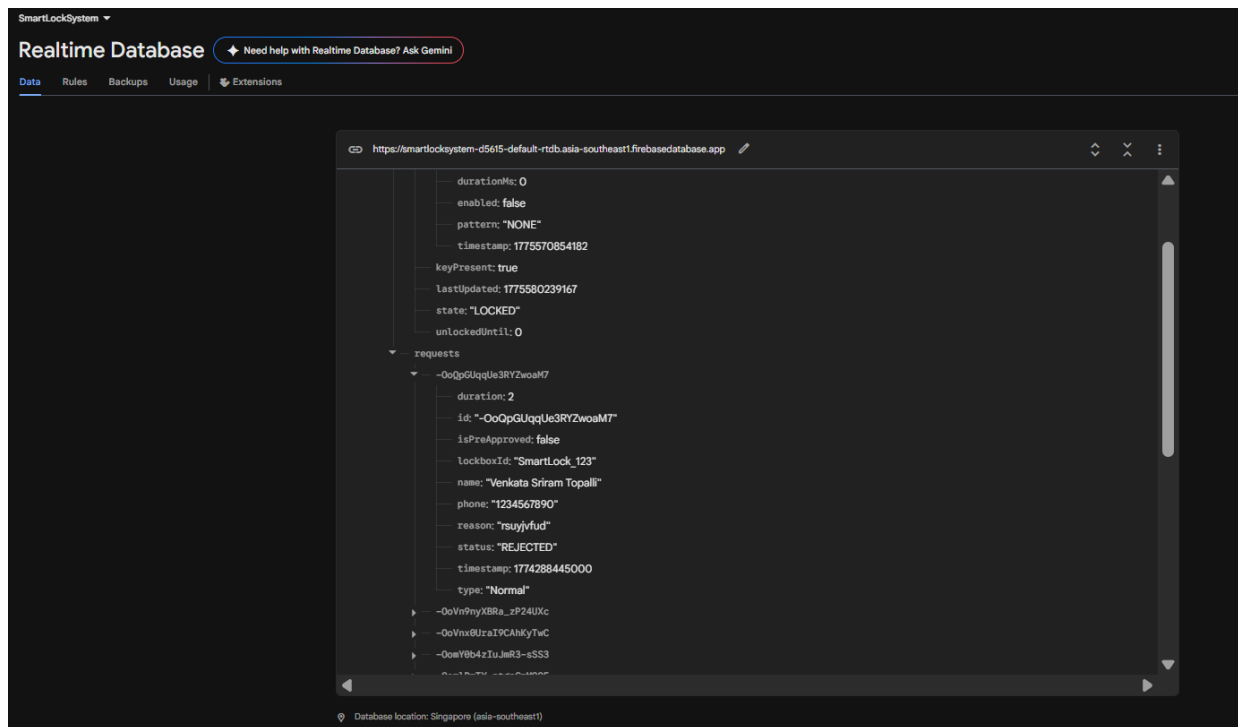
## APPENDIX A

### VISUAL PROOF OF IMPLEMENTATION AND USER INTERFACE

The following figures provide visual validation of the cyber-physical architecture, including the cloud database synchronization, the physical hardware assembly, and the frontend web portals utilized by both Owners and Guests.

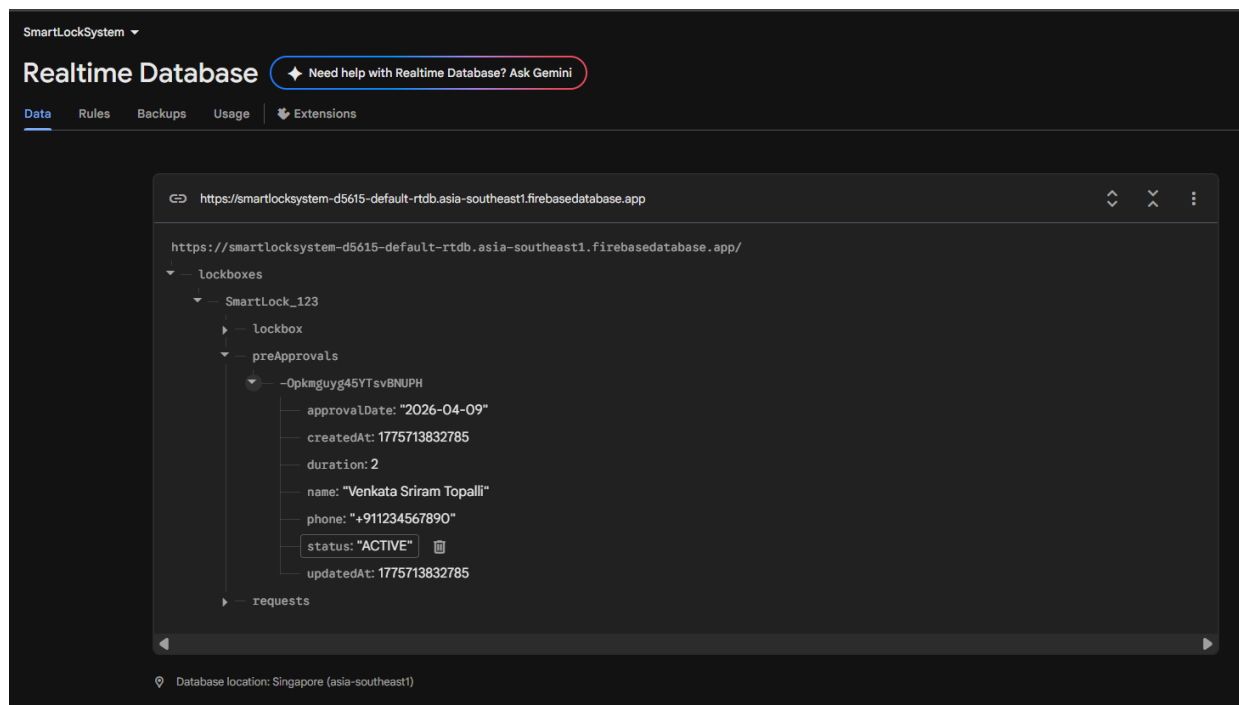


(a) Lockbox & Buzzer Nodes



(b) Requests Node (History Data)

Fig. 3. Firebase Realtime Database structure showing live synchronization and data storage (Part 1).



(c) Preapproval Node

Fig. 3. Firebase Realtime Database structure showing live synchronization and data storage (Part 2).

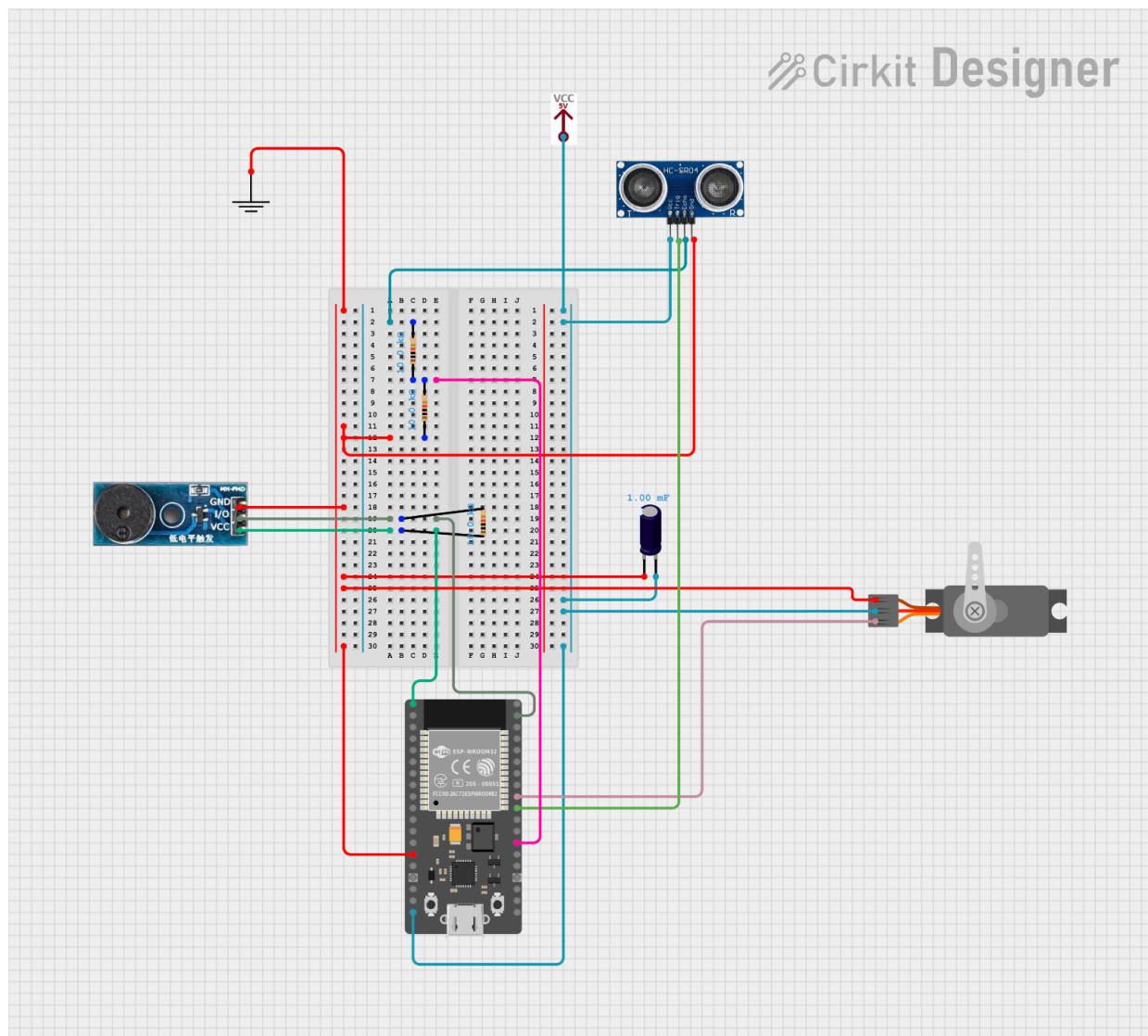


Fig. 4. Circuit schematic detailing the ESP32, sensor, and power management connections.

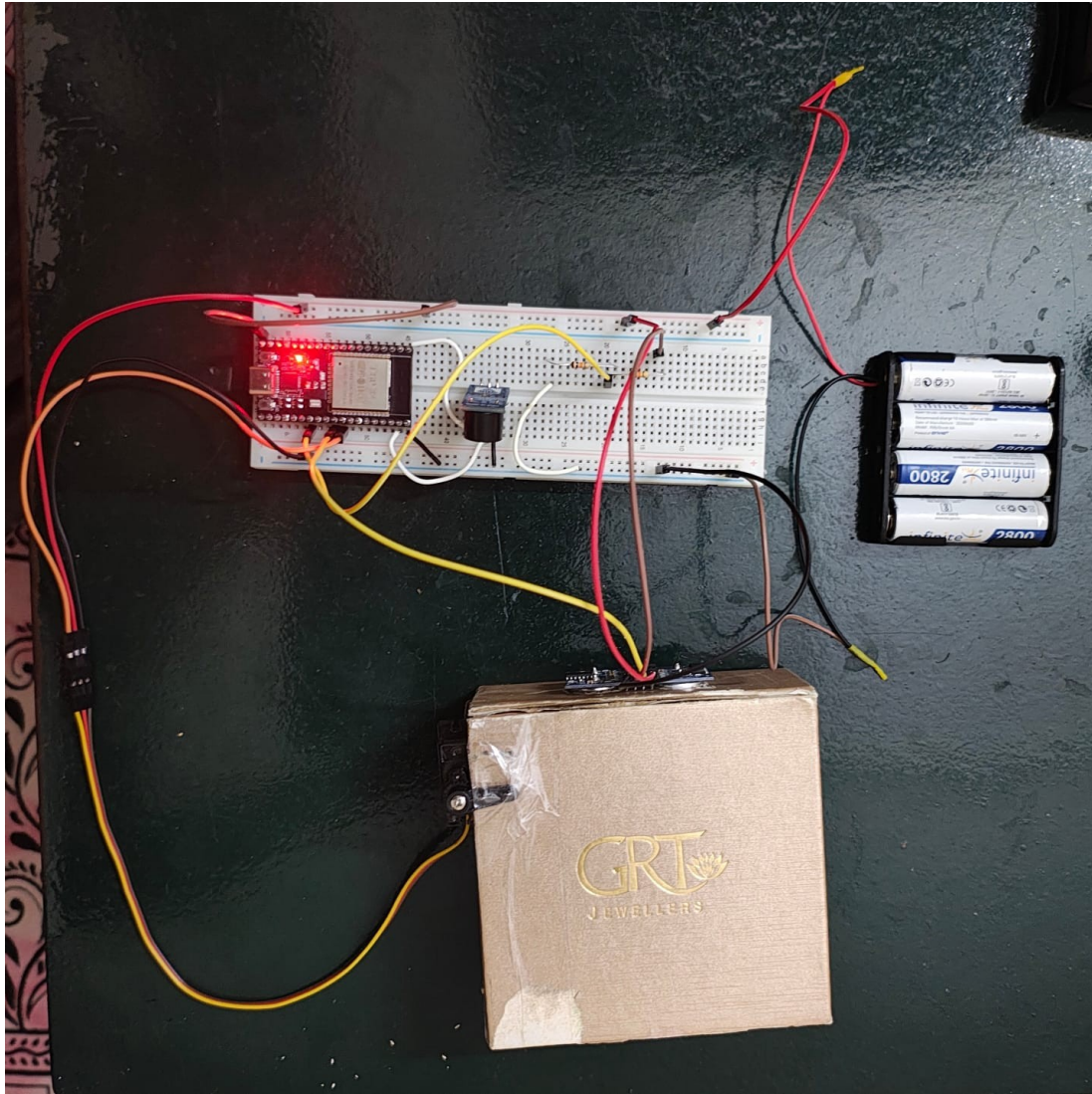
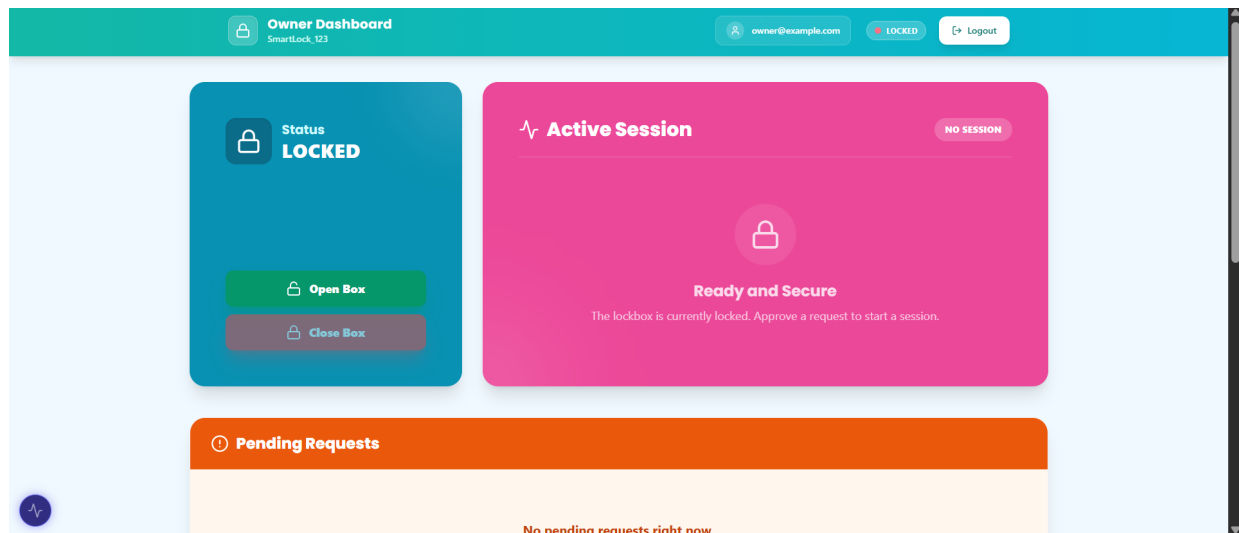
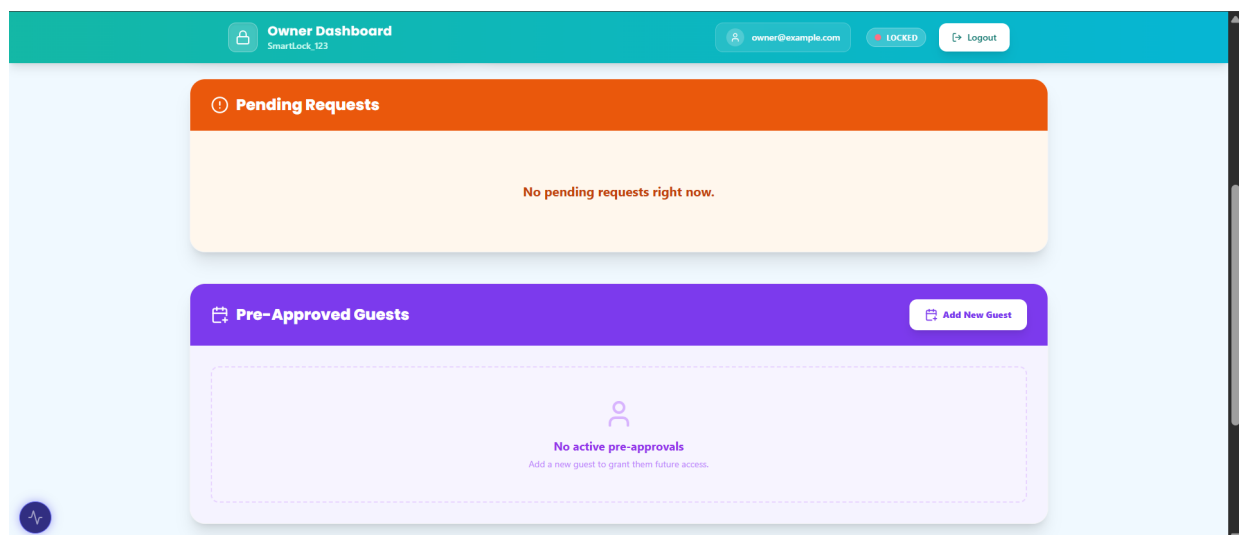


Fig. 5. Fully assembled Smart Lockbox hardware prototype.

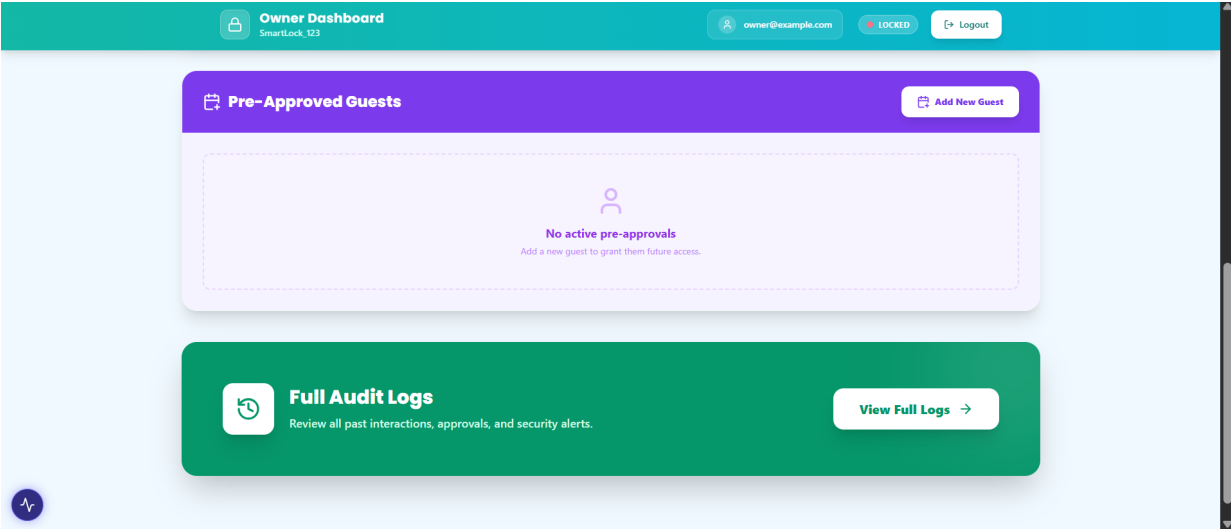


(a) Active Session Controls

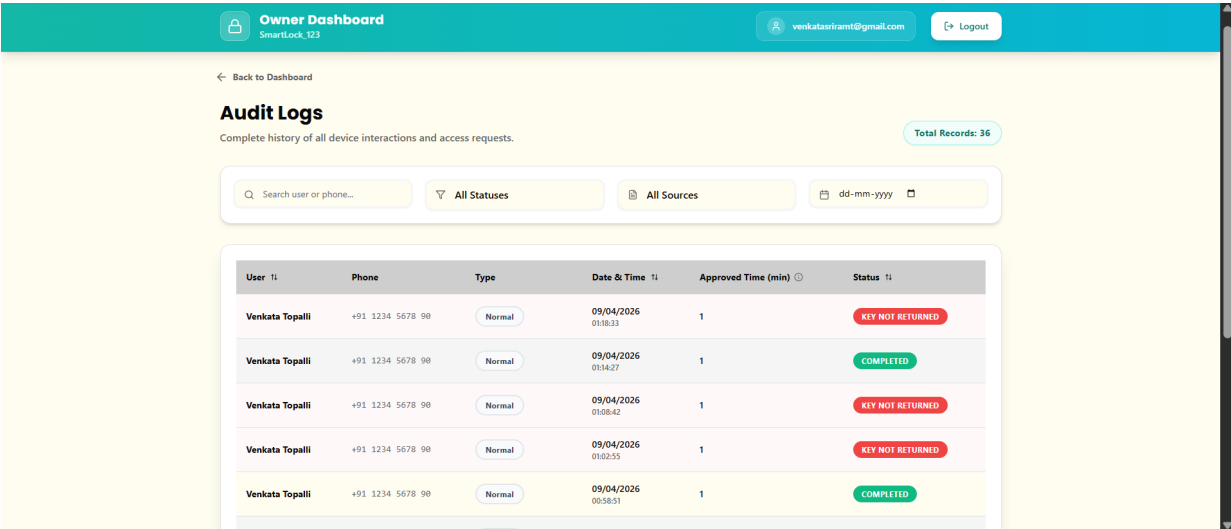


(b) Pending Requests View

Fig. 6. Owner Web Dashboard demonstrating session control, request management, and history logs (Part 1).

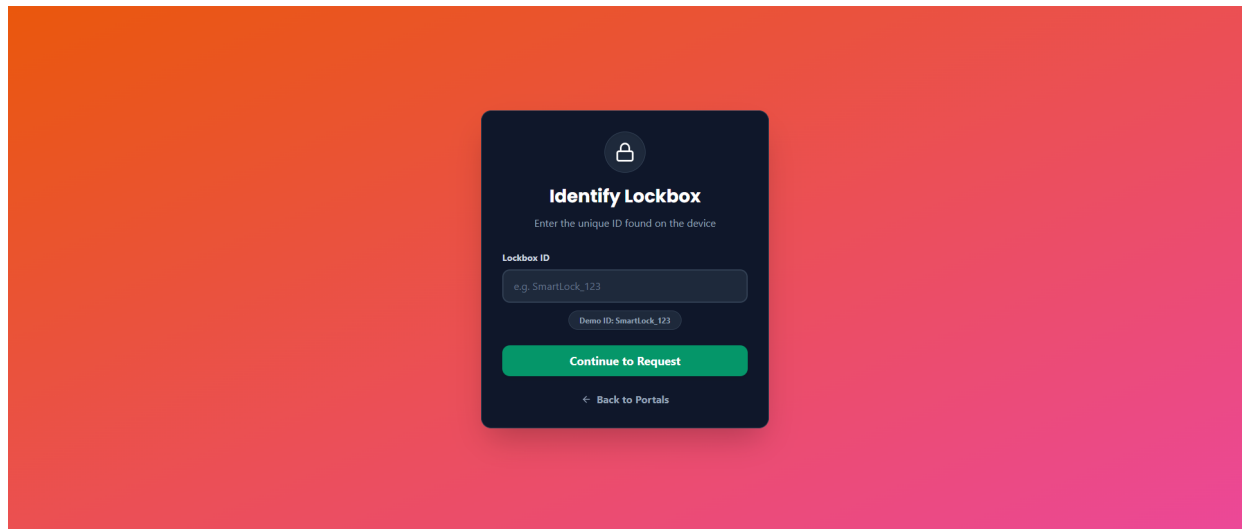


(c) Preapproved & History Menu



(d) Owner History Log

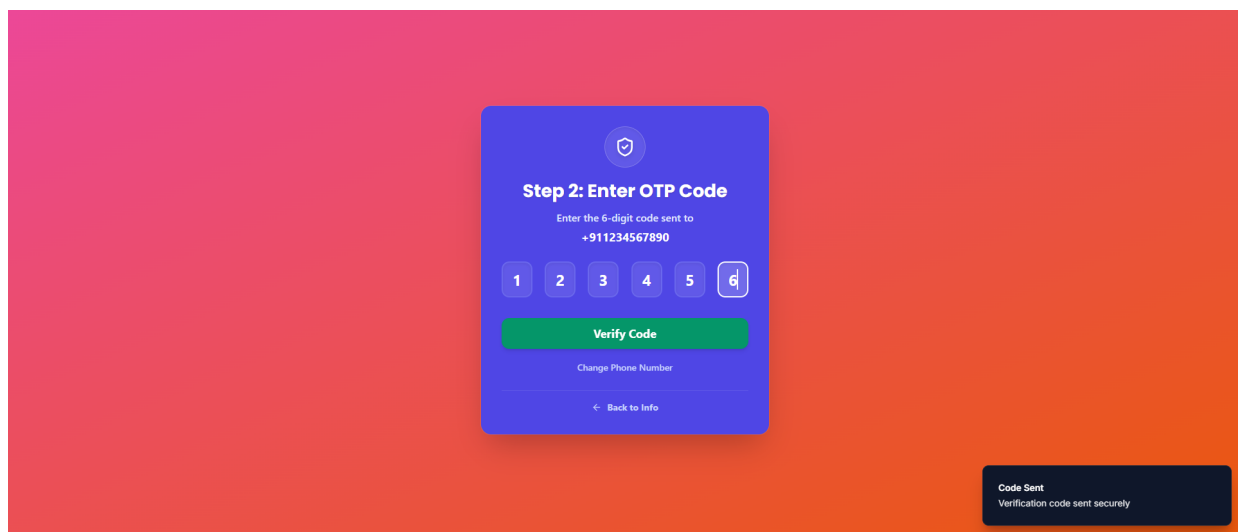
Fig. 6. Owner Web Dashboard demonstrating session control, request management, and history logs (Part 2).



(a) Lockbox ID Entry

(b) Personal Information Form

Fig. 7. Guest Entry Portal demonstrating the secure access request and verification workflow (Part 1).



(c) OTP Verification

Fig. 7. Guest Entry Portal demonstrating the secure access request and verification workflow (Part 2).